

Cidades ESG Inteligentes

Pipeline CI/CD, containerização com Docker e deploy automatizado em staging e produção

Integrantes

Julha Almeida Araujo, RM 562697

Walter Seixas Nogueira, RM 565533



Uma API que mede cidades pelos três pilares ESG

API REST em Java 21 e Spring Boot 4.1 que cadastra cidades, recebe medições de 9 indicadores e calcula um score ESG de 0 a 100 com ranking. Vale sempre a medição mais recente de cada indicador.



Ambiental, peso 40%

Emissões de CO₂ per capita, energia renovável e reciclagem de resíduos



Social, peso 30%

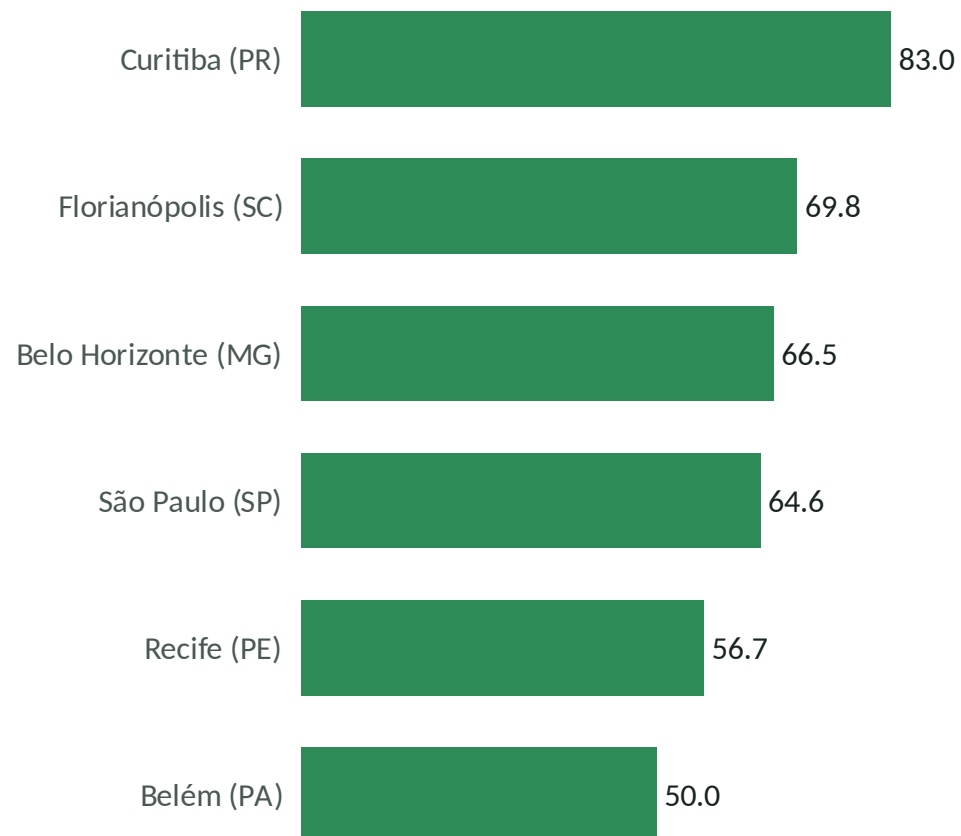
Coleta de esgoto, acesso a transporte público e mortalidade infantil



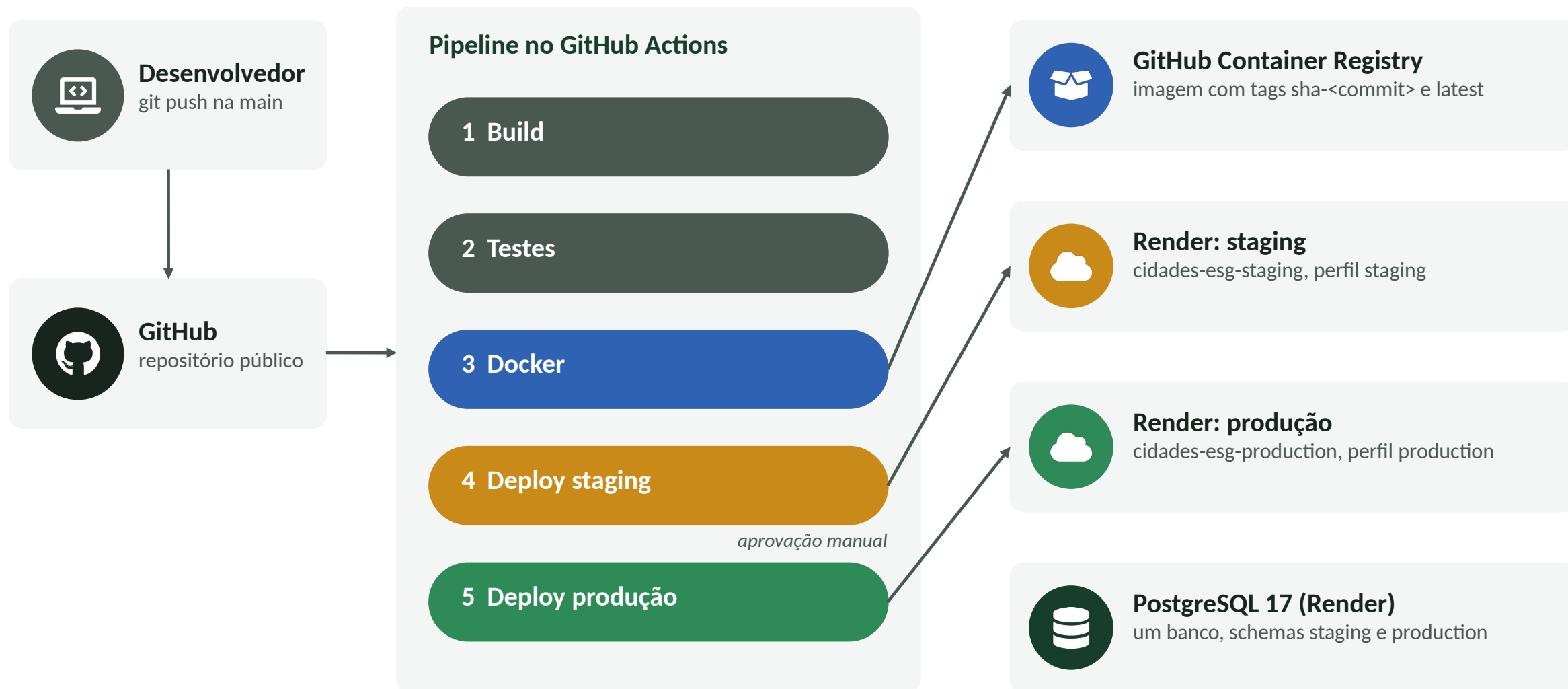
Governança, peso 30%

Transparência pública, serviços digitais e participação cidadã

Ranking ESG com os dados de demonstração (ilustrativos)

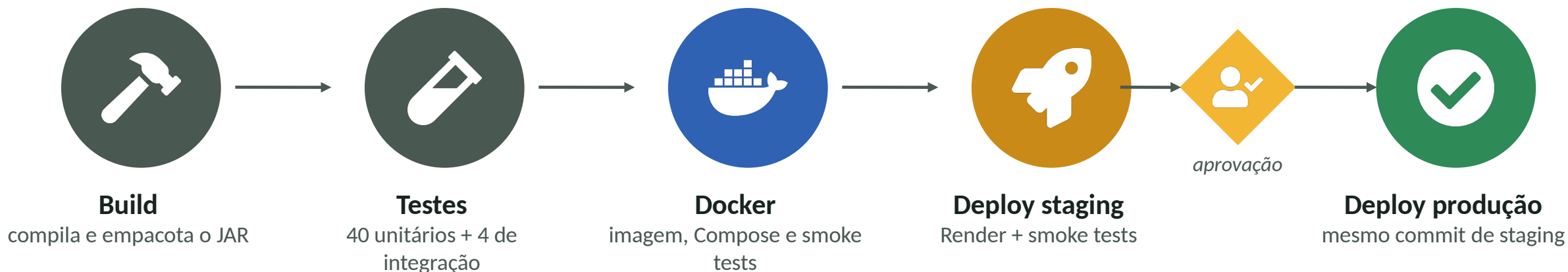


Arquitetura: do push à produção



Pipeline CI/CD com GitHub Actions

Ferramenta: GitHub Actions, arquivo `.github/workflows/ci-cd.yml`, com cinco jobs encadeados. Cada etapa só começa se a anterior passou.



Pull request para a main

Roda Build, Testes e Docker para validar a mudança antes do merge.
Não faz deploy.

Push na main ou execução manual

Fluxo completo: publica em staging, roda os smoke tests e, após a aprovação, publica o mesmo commit em produção.

Etapas do pipeline e critério para avançar

Etapa	O que acontece	Avança quando
Build	Java 21 (Temurin) com cache do Maven; ./mvnw package; o JAR fica guardado como artefato da execução.	O projeto compila e empacota
Testes	mvn verify: testes unitários (score, serviços com Mockito, API com MockMvc) e de integração com PostgreSQL 17 real (service container) e migrações Flyway. Publica relatório de testes e cobertura JaCoCo.	Todos os testes passam
Docker	Build da imagem multi-stage com cache; sobe app + PostgreSQL com Docker Compose e espera os health checks; roda smoke tests; publica no GHCR (sha-<commit> e latest).	Containers saudáveis e smoke tests OK
Deploy staging	Chama o deploy hook do Render com ref=<sha>, espera /api/info responder com o novo commit e roda smoke tests de leitura e escrita.	Staging no ar com o commit novo
Deploy produção	Após aprovação manual no ambiente production do GitHub, repete o deploy com o mesmo commit e roda smoke tests somente leitura.	Produção no ar com o mesmo commit

Lógica de deploy: o mesmo commit, verificado em cada ambiente

1

Deploy hook com o commit

O script render-deploy.sh chama o deploy hook do Render com ref=<sha>: o Render constrói exatamente aquele commit.

2

Troca sem indisponibilidade

O Render só envia tráfego para a nova versão quando /actuator/health/readiness (aplicação + banco) responde OK.

3

Confirmação da versão

O pipeline consulta /api/info até o commit novo aparecer, com tempo limite de 20 minutos e erro claro se falhar.

4

Smoke tests

Staging: leitura e escrita (cria, pontua e remove uma cidade). Produção: somente leitura.

5

Aprovação para produção

O ambiente production do GitHub exige um revisor; só então o mesmo commit vai para produção.



Segredos e configuração

GitHub Secrets

RENDER_DEPLOY_HOOK_STAGING e
RENDER_DEPLOY_HOOK_PRODUCTION

GitHub Variables

STAGING_URL e PRODUCTION_URL

GitHub Environments

staging e production, com histórico de deploys e revisores obrigatórios

Render

senha do banco injetada pela plataforma; nada sensível no código

Docker: imagem multi-stage, enxuta e segura



Etapa 1: build

`eclipse-temurin:21-jdk-alpine`

- Copia pom.xml e Maven Wrapper e baixa as dependências (camada em cache)
- Copia o código e gera o JAR com `./mvnw package`
- Separa o JAR em camadas com `jarmode=tools extract --layers`
- Não roda os testes de novo: eles já passaram na etapa Testes do pipeline



Etapa 2: runtime (imagem final)

`eclipse-temurin:21-jre-alpine`

application código da API (muda a cada commit)

snapshot-dependencies dependências SNAPSHOT

spring-boot-loader inicializador do Spring Boot

dependencies bibliotecas (mudam pouco)

As camadas chegam da etapa 1 por `COPY --from=build`. Um deploy novo reenvia só a camada `application`.



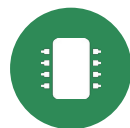
Usuário sem root

a aplicação roda como app



HEALTHCHECK

liveness a cada 30 s



JVM para 512 MB

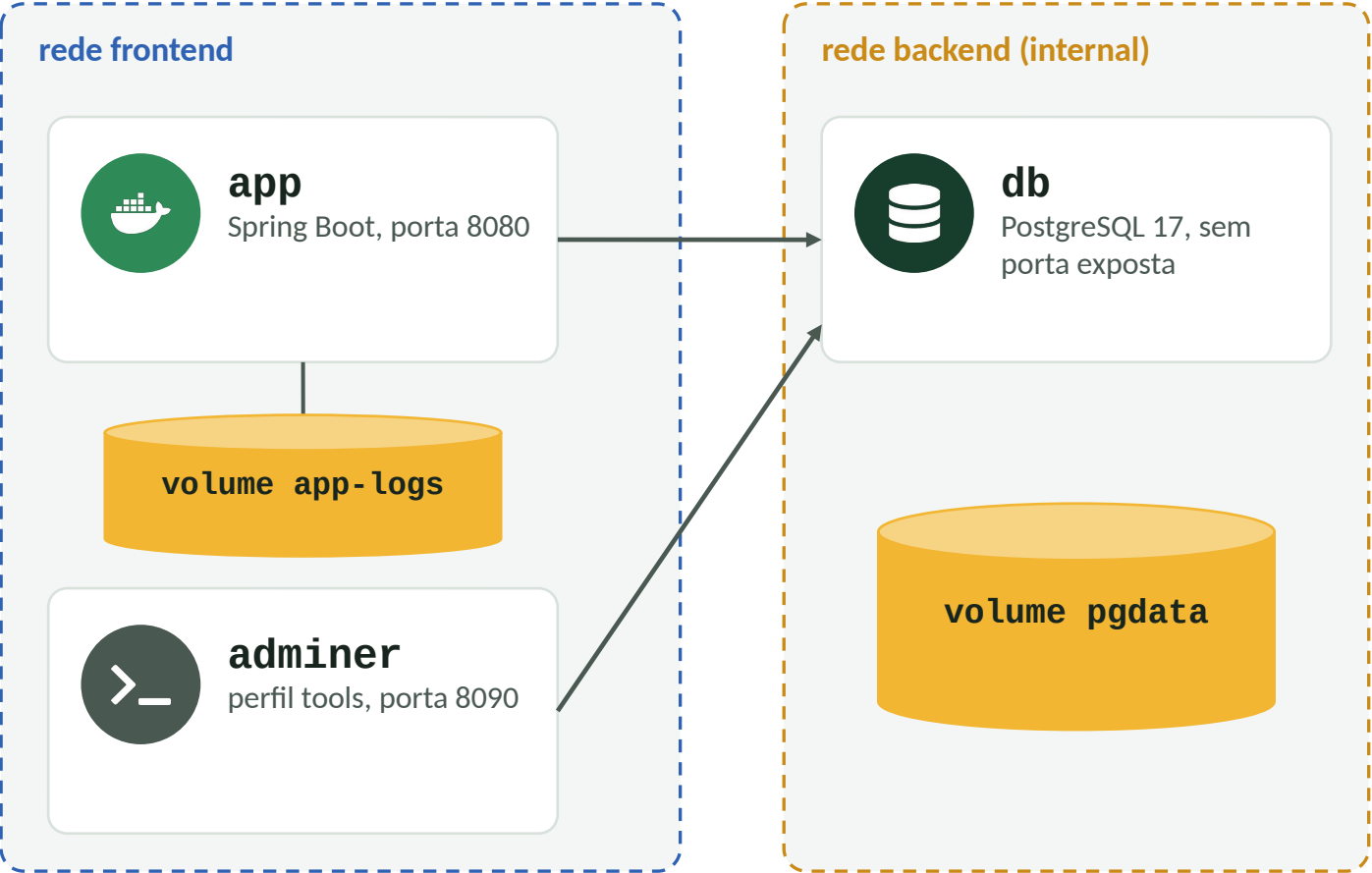
heap 60%, GC serial, C1



Desligamento gracioso

exec java + shutdown graceful

Docker Compose: aplicação, banco, volumes e redes



Serviços	app, db e adminer (opcional)
Volumes	pgdata (dados persistentes) e app-logs (arquivo de log)
Redes	frontend com portas publicadas; backend interna, sem acesso externo
Variáveis	arquivo .env (modelo .env.example); senha obrigatória
Ambientes	o mesmo arquivo sobe staging (8081) e produção (8082) isolados com -p e --env-file

Healthchecks: o app só sobe quando o db está saudável (depends_on: service_healthy) e fica pronto quando /actuator/health/readiness responde.

Comandos usados e imagem criada

```
# construir a imagem
docker build -t cidades-esg-inteligentes .
# subir aplicação + PostgreSQL
cp .env.example .env
docker compose up -d --build
docker compose ps
docker compose logs -f app
# testar a API de ponta a ponta
bash scripts/smoke-test.sh \
    http://localhost:8080 local --escrita
# simular staging e produção lado a lado
docker compose -p cidades-esg-staging --env-file .env \
    --env-file envs/staging.env up -d --build
# parar (com -v apaga os dados)
docker compose down -v
```



Imagem criada

Registro

ghcr.io/julhaalmeida/
cidades-esg-inteligentes

Tags

sha-<commit> (versão rastreável) e latest

Base

eclipse-temurin:21-jre-alpine

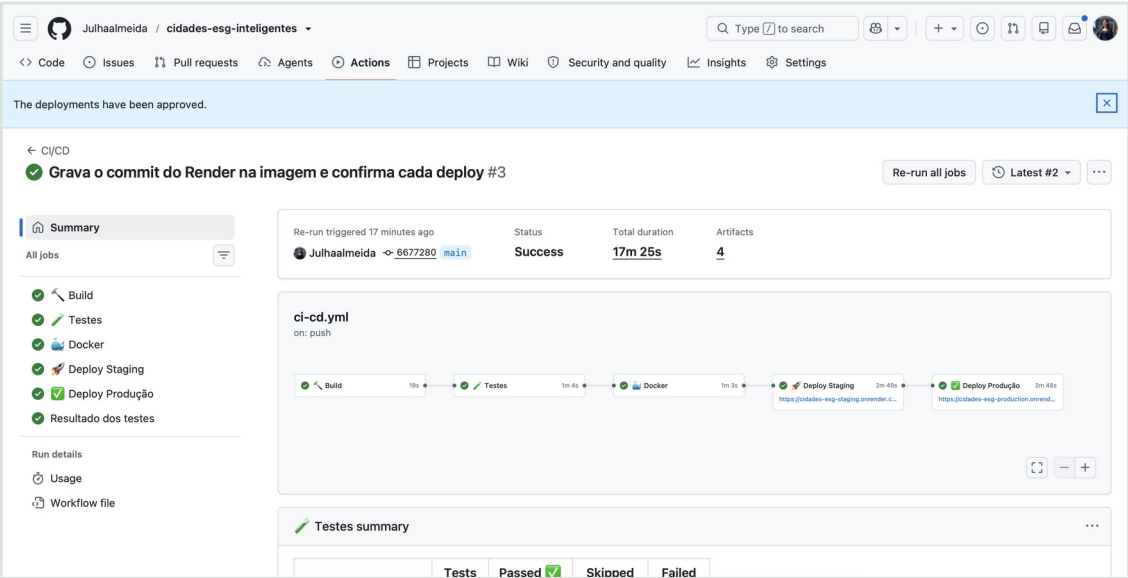
Execução

usuário app (sem root), porta 8080, health check
em /actuator/health/liveness

Rastreabilidade

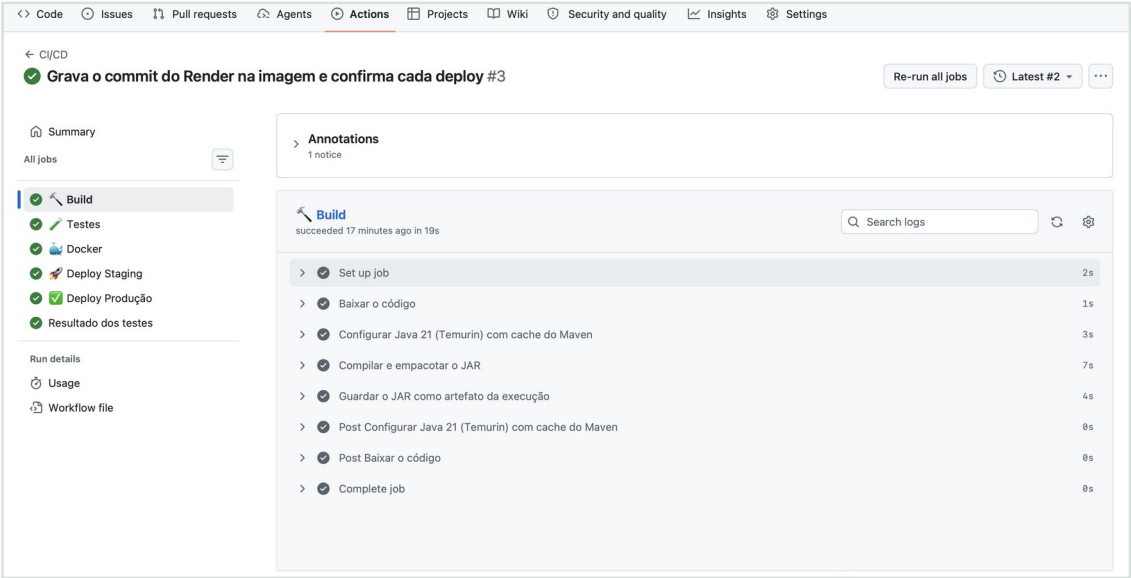
o commit vai para a imagem (APP_COMMIT) e
aparece em /api/info

Pipeline em execução: visão geral e build



Pipeline completo

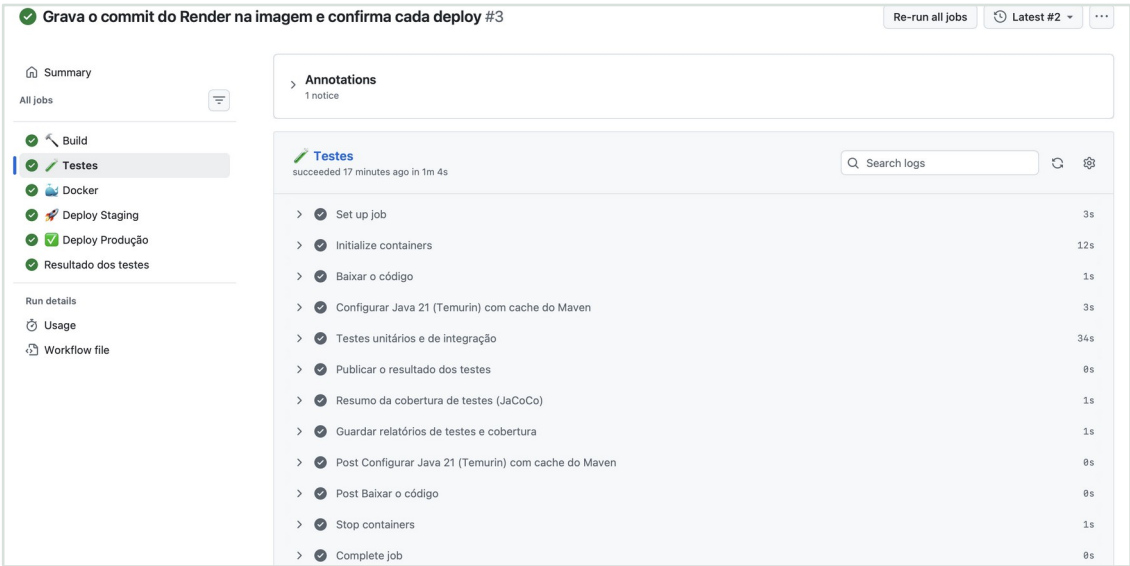
Execução #3 na main, commit 6677280: os cinco jobs concluídos com sucesso



Build

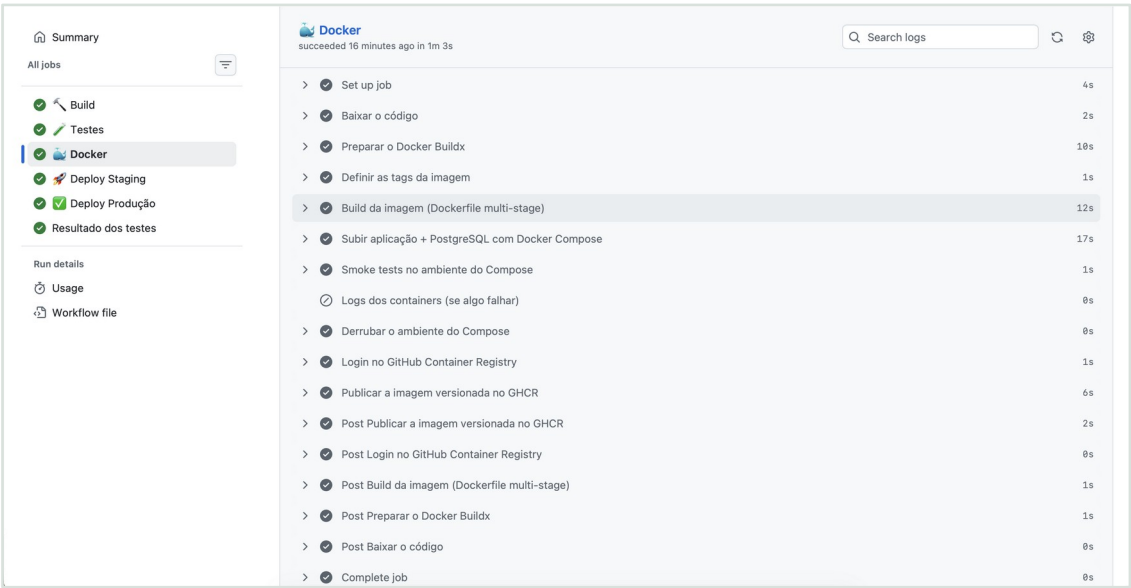
Job Build: Java 21 com cache do Maven, JAR compilado e guardado como artefato

Testes automatizados e validação com Docker



Testes

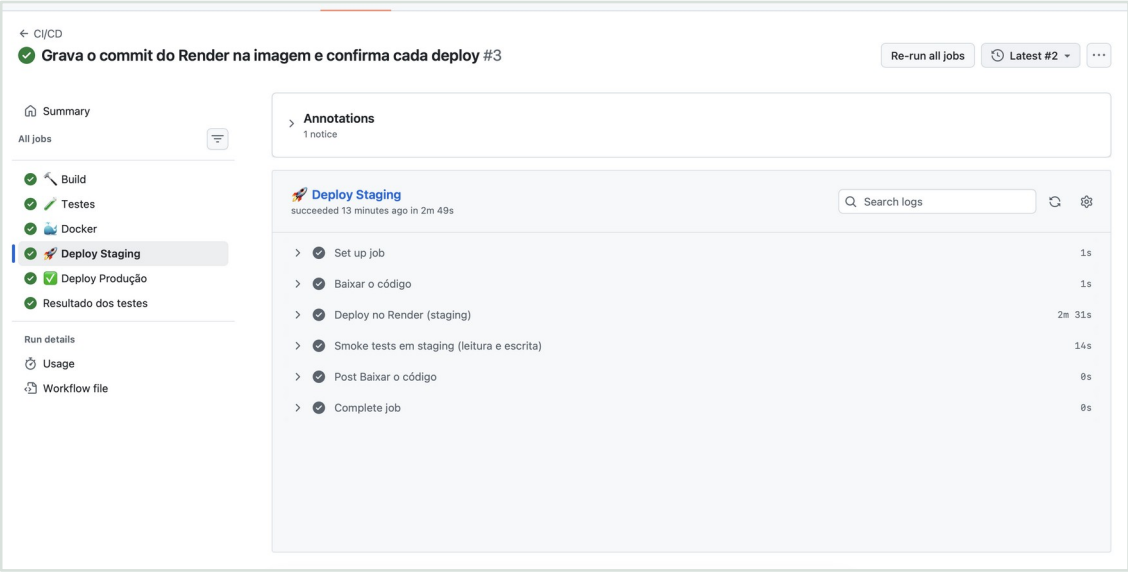
Testes unitários e de integração com PostgreSQL, relatório publicado e cobertura (JaCoCo)



Docker no CI

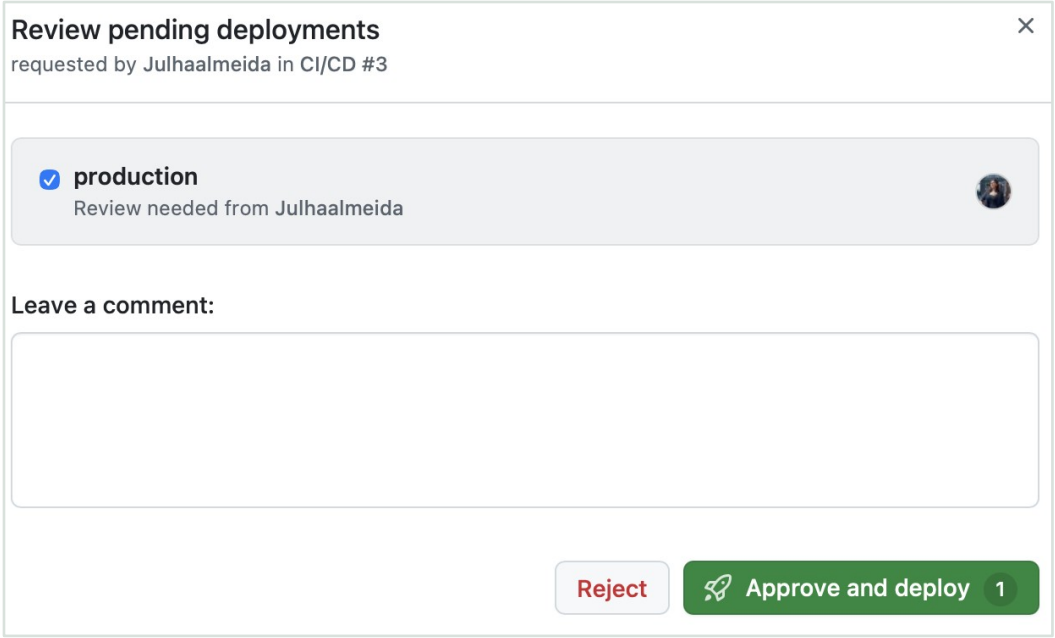
Build da imagem, Compose com PostgreSQL, smoke tests e publicação no GHCR

Deploy em staging e aprovação para produção



Deploy em staging

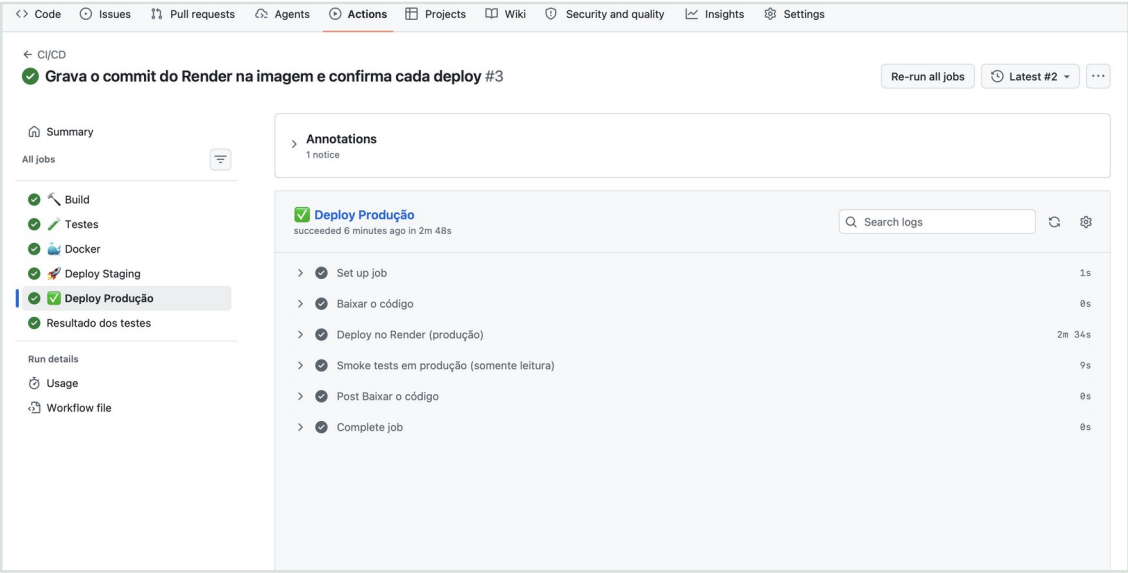
Deploy no Render com o commit do pipeline e smoke tests de leitura e escrita



Aprovação

Revisão obrigatória do ambiente production antes do deploy

Deploy em produção e imagem publicada



Deploy em produção

O mesmo commit de staging publicado em produção, com smoke tests somente leitura

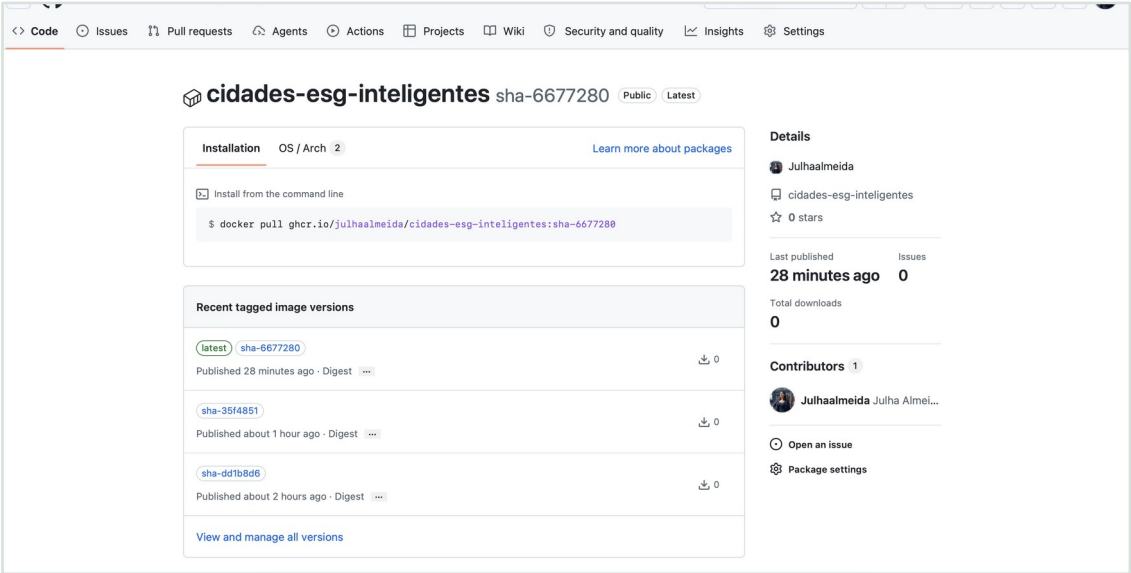
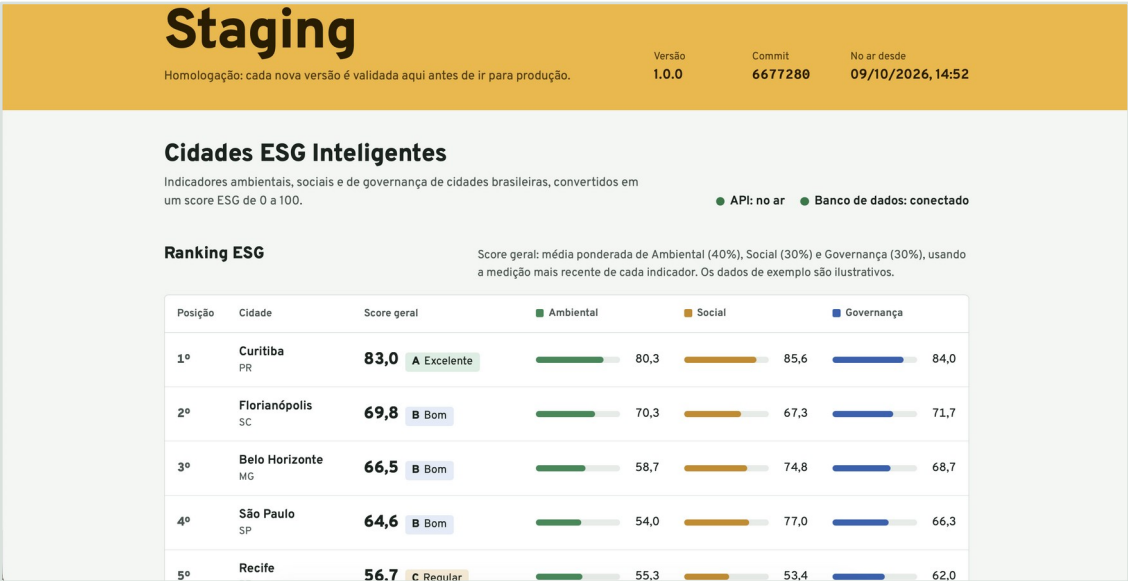


Imagem no GHCR

ghcr.io/julhaalmeida/cidades-esg-inteligentes, com as tags sha-6677280 e latest

Ambientes funcionando: staging e produção



Staging

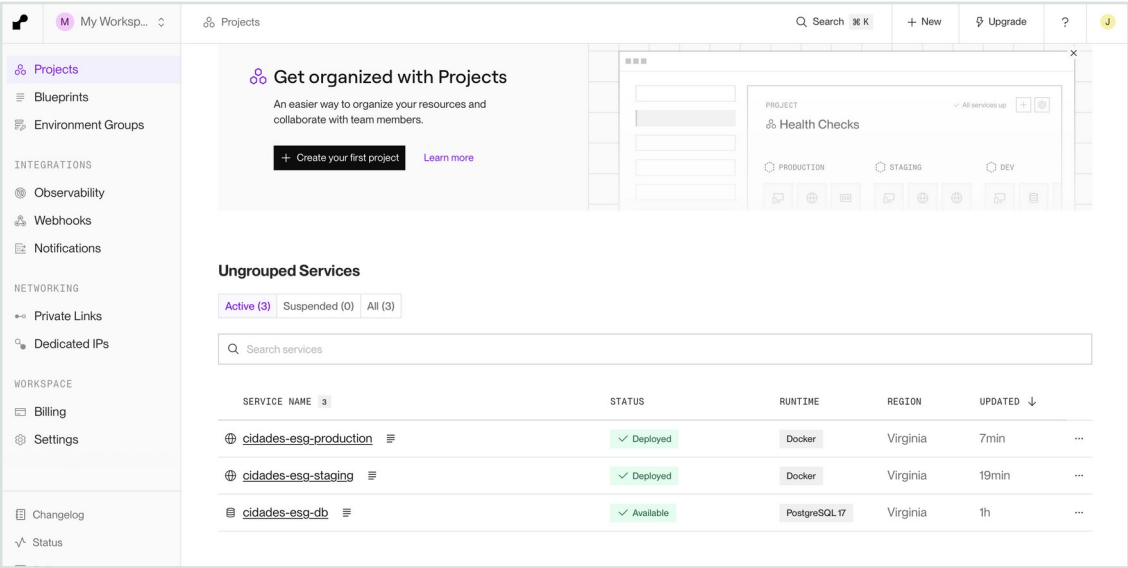
cidades-esg-staging.onrender.com: faixa amarela, versão 1.0.0 e commit 6677280



Produção

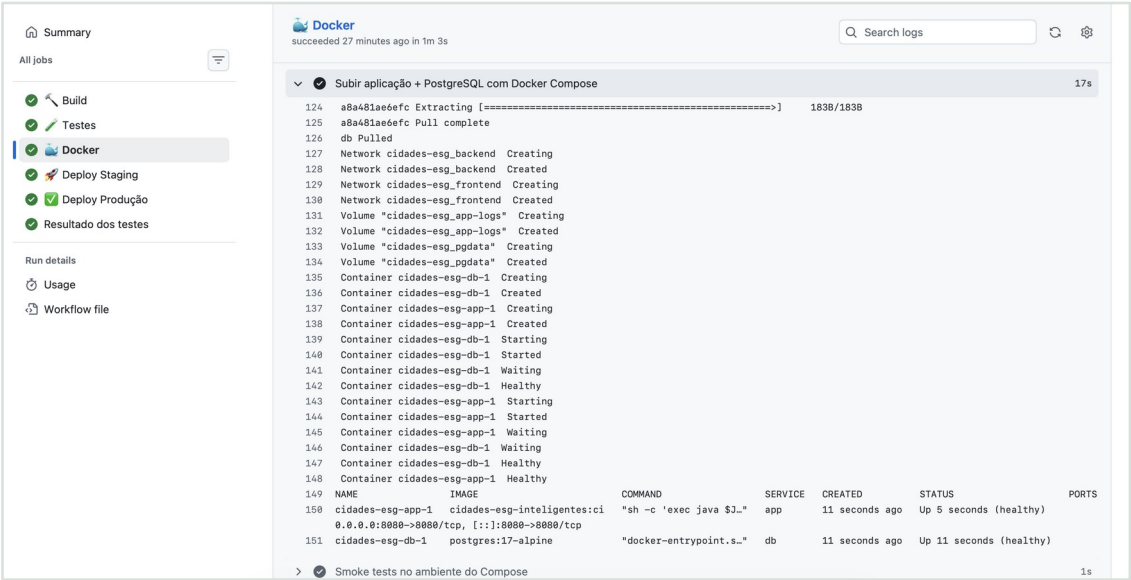
cidades-esg-production.onrender.com: faixa verde e o mesmo commit 6677280

Infraestrutura no Render e Docker Compose



Render

Serviços Docker de staging e produção e o banco PostgreSQL 17, criados pelo render.yaml



Docker Compose

No pipeline: redes frontend e backend, volumes pgdata e app-logs e containers saudáveis

Desafios encontrados e como resolvemos



Só 1 banco gratuito no Render

Cada ambiente usa um schema próprio (DB_SCHEMA), criado e migrado pelo Flyway.



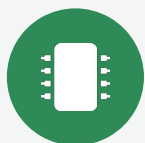
Produção igual a staging

O mesmo commit vai para os dois deploy hooks (ref=<sha>) e é conferido em /api/info.



Commit não chegava no Render

O Render só repassa o commit para ARGs declarados. Declaramos ARG RENDER_GIT_COMMIT no Dockerfile.



512 MB e 0,1 CPU

JVM ajustada para container, pool de conexões pequeno e imagem JRE Alpine.



Testar com o banco real

PostgreSQL como service container nos testes e Compose com smoke tests no CI.



Segredos fora do código

GitHub Secrets e Variables, .env fora do Git e senha do banco injetada pelo Render.

Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	✓
Dockerfile funcional	✓
docker-compose.yml ou arquivos Kubernetes	✓
Pipeline com etapas de build, teste e deploy	✓
README.md com instruções e prints	✓
Documentação técnica com evidências (PDF ou PPT)	✓
Deploy realizado nos ambientes staging e produção	✓



7 de 7 itens

Arquivos no .zip, README com prints e esta documentação

Links do projeto



Repositório

<https://github.com/Julhaalmeida/cidades-esg-inteligentes>



Staging

<https://cidades-esg-staging.onrender.com>



Produção

<https://cidades-esg-production.onrender.com>



Swagger (produção)

<https://cidades-esg-production.onrender.com/swagger-ui.html>

Obrigado!

Julha Almeida Araujo, RM 562697
Walter Seixas Nogueira, RM 565533